OpenGov

Salesforce Developer Assessment

Addendum: Real-Time Toasts via Platform Events

Server-to-UI Event Bridge · empApi · Loose Coupling

1. Why a Platform Event Here?

When a sales rep contracts a Quote, the Record-Triggered Flow runs server-side and creates Assets. Without an event bridge, the rep would need to manually refresh the page to know whether the automation succeeded - or rely on Chatter notifications, which are easy to miss.

A Platform Event closes this loop: the Flow publishes an event the moment the work is done; a headless LWC subscribed to the event channel dispatches a native Salesforce toast in real time on whatever page the rep is on. No polling, no manual refresh, no missed feedback.

Alternatives Considered

Alternative	Why Not Chosen
Page refresh after save	Janky UX. Forces a full page reload. The user loses their scroll position and any unsaved work elsewhere. Also fails if the user clicks away before the Flow completes.
Client polls via setInterval	Wasteful network traffic. Latency is whatever the poll interval is - typically 5-10 seconds. Hits Apex CPU and API limits at scale.
Chatter notification only	Easy to miss. The notification badge is small and reps often have it muted. Also slow - Chatter posts take 1-3 seconds to surface.
Email alert	Way too slow. Email arrives minutes later. Useless for real-time feedback on a save action.
Platform Event (chosen)	Sub-second delivery. Loose coupling - any subscriber can react. Cross-tab and cross-user delivery. Built on Salesforce CometD - same channel the platform uses internally for CDC.

2. Event Schema: Asset_Creation_Result__e

The Platform Event is intentionally narrow - only the fields a subscriber needs to render a meaningful toast and route it to the correct user. Following the principle: events should carry just enough context to be actionable, not full record state.

Field	Type	Purpose
Quote_Id__c	Text(18)	Salesforce ID of the Quote the work was performed on. The subscriber LWC matches this against its own recordId to filter out events for other Quotes.
Status__c	Text(18)	Success if Assets were created cleanly. Error/NoHardware on fault. Drives toast variant (Success, Error, or NoHardware.)
Asset_Count__c	Number(5,0)	Count of Assets actually created. Allows the toast to be specific: "3 assets created" not just "done".
Message__c	Text(255)	Human-readable message. For success: confirmation text. For fault: the cleaned error message from FlowErrorHandler. Same text that posts to Chatter.

3. End-to-End Architecture

The full publish-subscribe pipeline runs in three layers, and each layer is independently testable:

3.1 Publisher: The Flow

- After Create Records (success path), the Flow runs a 'Create Records' on Asset_Creation_Result__e with Status__c = Success, Asset_Count__c set to the loop counter, Message__c = success text, and Quote_Id__c = $Record.Id.
- On the fault path - after FlowErrorHandler has cleaned the message - the same Platform Event is published with Status__c = Error and Message__c = the cleaned error. Failure feedback is just as actionable as success feedback.
- Publishing a Platform Event is itself a DML operation, so it respects the standard governor limits. One publish per Flow execution is well within bounds.

3.2 Channel: /event/Asset_Creation_Result__e

- Salesforce provides the streaming channel automatically. No middleware to deploy.
- Events are delivered via CometD - the same protocol Salesforce uses for Change Data Capture, Generic Streaming, and PushTopics.
- The platform handles retry semantics: subscribers that disconnect and reconnect within the retention window receive missed events.

3.3 Subscriber: The assetCreationToast LWC

- A 'headless' LWC - meaning it renders nothing visible. It exists purely to listen and react.
- Uses the lightning/empApi module to subscribe to /event/Asset_Creation_Result__e on connectedCallback.
- On each incoming event: checks if the event's Quote_Id__c matches this component's @api recordId. If not, ignores it - the user is on a different Quote.
- On match: dispatches a ShowToastEvent with variant inferred from Success__c (success | warning | error).
- Cleanup: unsubscribes on disconnectedCallback to prevent memory leaks.

4. Why This Pattern Matters

4.1 Loose Coupling

The Flow doesn't know about the LWC. The LWC doesn't know about the Flow. They communicate only through the event schema. That means:

- I can add a SECOND subscriber - say, an Apex trigger that logs every event to a Custom Log Object - without modifying the publisher.
- I can change the publisher's internal implementation (refactor the Flow into Queueable Apex, for example) without modifying the subscriber. The contract is the event schema.
- External systems can subscribe to the same channel via the Streaming API. ERP, billing, or provisioning systems can react to Asset creation in real time without us writing integration code.

4.2 Real-Time UX without Polling

Polling has well-known costs: client CPU, network traffic, Apex governor consumption, and latency equal to the polling interval. CometD push gives sub-second delivery with zero polling. This is the same pattern Salesforce uses for the Lightning bell-icon notifications, Change Data Capture, and Live Agent chat presence.

4.3 Where Else This Pattern Applies

- Long-running async jobs: 'Your data export is ready' toast when a Batch Apex job completes.
- Cross-user collaboration: Sales rep contracts a Quote; the Service team's queue page gets a real-time alert without refresh.
- Approval routing: requester sees a toast the moment an approver acts on their request.
- External system callbacks: a webhook from a payment processor fires a Platform Event, and the user sees the result in their browser within a second.

5. Production Considerations

Concern	How This Implementation Handles It
Event delivery limits	Salesforce allows 250K event deliveries per 24 hours on Developer Edition (much higher for paid editions). Our event volume is ≈ 1 per contracted Quote - well below limits.
Subscriber memory leaks	LWC calls unsubscribe() in disconnectedCallback. Without this, navigating between Quote pages would accumulate stale subscriptions. Explicit cleanup is mandatory.
Event ordering	Platform Events are ordered per channel but not globally. For our use case (one event per Quote save), ordering doesn't matter. For high-frequency channels, consumers would need to handle out-of-order delivery.
Replay after disconnect	We subscribe with replay ID -1 (only NEW events). For mission-critical use cases, we'd use replay ID -2 (resume from last known position) and persist the last received replay ID in localStorage.
Error handling	onError() is wired and currently logs to console. Production would route those errors to a structured log (Custom Log Object) to surface streaming failures.
Sticky vs auto-dismiss	Success toasts auto-dismiss after 3 seconds (default). Error toasts use mode:'sticky' so the user must manually dismiss them - prevents accidentally missing critical failures.